

MLOps Agéntico: Orquestación Autónoma de Clústeres de Entrenamiento de Visión Distribuida Impulsada por LLM usando Model Context Protocol (MCP)

William Steve Rodriguez Villamizar (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1 Resumen & Palabras Clave

Resumen: Las arquitecturas tradicionales de Operaciones de Machine Learning (MLOps) sufren de cuellos de botella operativos severos al escalar cargas de trabajo de visión por computadora distribuidas. Presentamos un marco de investigación aplicada utilizando el Protocolo de Contexto de Modelo (MCP) para conectar Grandes Modelos de Lenguaje (LLMs) y clústeres físicos de GPU. Al aislar los nodos del clúster mediante un patrón Invocador-Ejecutor a través de demonios Celery, mitigamos efectivamente los fallos catastróficos de Fuera de Memoria (OOM) que frecuentemente bloquean los procesos host durante sesiones intensas de entrenamiento YOLO. Además, integramos un mecanismo de validación de datos "shift-left" para rechazar preventivamente conjuntos de datos corruptos montados en red antes de asignar memoria de la GPU. Nuestras evaluaciones empíricas demuestran que este enfoque redujo la latencia de orquestación en un 43%, disminuyó el consumo pico de memoria de un inestable 28GB a un límite estricto de 16GB, y previno por completo las caídas por OOM durante una prueba de estrés de 72 horas. La integración de LLMs como gestores autónomos del clúster demuestra un ejemplo concreto de investigación aplicada que produce resultados robustos y reproducibles para la comunidad de ingeniería de ML.

Palabras Clave: MLOps Agéntico, Model Context Protocol, Computación Distribuida, Orquestación LLM, Validación Shift-Left.

2 Información del Autor

Esta investigación fue conceptualizada y desarrollada por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para el ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3 Introducción

Escalar clústeres de entrenamiento distribuido para modelos de visión por computadora de alta resolución presenta desafíos de ingeniería severos. Las fugas de memoria latentes y los scripts de programación frágiles degradan rutinariamente el rendimiento del clúster. Los investigadores encuentran frecuentemente caídas silenciosas por Fuera de Memoria (OOM) donde el demonio de entrenamiento principal asigna memoria más allá de los límites físicos de la GPU, bloqueando todo el nodo y requiriendo reinicios forzados manuales. Esta monopolización del hardware limita directamente la escalabilidad de los pipelines de Deep Learning automatizados.

Abordamos estos puntos de dolor específicos descentralizando la arquitectura de cómputo e introduciendo un paradigma de MLOps Agéntico. Equipamos un Gran Modelo de Lenguaje (LLM) con herramientas especializadas del Protocolo de Contexto de Modelo (MCP), otorgándole la capacidad de monitorear dinámicamente la salud del nodo, validar conjuntos de

datos y despachar trabajos de entrenamiento aislados. A diferencia de la orquestación YAML estática convencional, este enfoque permite que el agente razone sobre el estado actual del clúster y enrute adaptativamente las cargas de trabajo hacia nodos saludables.

Aislamos la ejecución real del entrenamiento dentro de contenedores Docker efímeros gestionados por una cola de tareas Celery, introduciendo el patrón Invocador-Ejecutor. Este límite físico asegura que si un script de entrenamiento YOLO filtra memoria, solo el contenedor aislado muere, dejando el demonio anfitrión completamente intacto.

4 Trabajo Relacionado

La convergencia de agentes autónomos y la ingeniería de ML se ha acelerado rápidamente. Smith et al. [10] demostraron que el aprendizaje por refuerzo permite que los agentes asuman tareas básicas de ingeniería de ML, aunque su enfoque carecía de aislamiento de hardware físico. Doe et al. [3] propusieron sistemas multiagente para AutoML de canal completo, pero dependían de programadores centralizados susceptibles a fallas de un solo punto.

Los marcos AgentOps [6] han intentado resolver los desafíos de monitoreo conectándose a la ventana de contexto del LLM. Sin embargo, ninguno de estos enfoques aborda la degradación específica del hardware causada por cargas masivas de trabajo de visión por computadora. Nuestro trabajo se basa en la base teórica de Liu et al. [7] sobre el Protocolo de Contexto de Modelo, extendiéndolo específicamente para interactuar con demonios de GPU respaldados por Celery. Diferenciamos nuestro enfoque imponiendo un estricto mecanismo de validación shift-left [2] antes de que cualquier instrucción del LLM llegue a los nodos de cómputo. Investigación adicional por Kim y Park [5] investigó la mitigación de OOM usando cgroups de Linux, lo que inspiró fuertemente nuestra estrategia de contenedores efímeros. La CLI fundacional wisrovisuit [9] sentó las bases para esta arquitectura, proporcionando los conjuntos de herramientas deterministas necesarios para LLMOps [11] y la orquestación generativa autónoma [8]. Finalmente, las optimizaciones de Celery para entornos de alto rendimiento [4] y el

impacto ambiental de una programación eficiente [1] influyeron fuertemente en nuestro diseño del broker.

5 Arquitectura Propuesta / Metodología

Nuestro sistema desacopla la orquestación lógica de la ejecución física. La arquitectura consta de tres capas principales: la Interfaz LLM-MCP, el Gateway Invocador, y el Ejecutor Efímero.

5.1 La Interfaz LLM-MCP

Expusimos la API del clúster a través de un servidor personalizado de Model Context Protocol. El LLM actúa como el cliente, recibiendo indicaciones en lenguaje natural del usuario (ej., "Entrenar un modelo YOLOv10 en el dataset custom-defect"). El servidor MCP traduce las llamadas a herramientas del LLM en cargas útiles REST concretas y las despacha asincrónicamente vía `httpx` al Gateway FastAPI de NeuralForge. Esto desacopla al agente de las colas brutas de Celery y elimina la necesidad de que los investigadores escriban frágiles scripts de Bash.

5.2 Validación de Datos Shift-Left

Antes de despachar cualquier trabajo, el LLM activa una herramienta de validación estática. Esta herramienta monta las unidades de red (CIFS/Samba) y verifica la integridad de los encabezados de las imágenes y las anotaciones de los cuadros delimitadores. Formalizamos la restricción de validación como:

$$V(D) = \prod_{i=1}^N \delta(H_i) \cdot \delta(B_i) \quad (1)$$

donde H_i representa la integridad del encabezado de la imagen i , y B_i representa la validez de las coordenadas del cuadro delimitador. Si $V(D) = 0$, el conjunto de datos D es rechazado. Al detectar archivos rotos o faltantes en el borde del canal (shift-left), evitamos la asignación de memoria de la GPU a procesos inevitablemente condenados.

5.3 El Patrón Invocador-Ejecutor

Una vez validado, el servidor MCP encola la tarea a través de la API en un broker Celery distribuido (Redis/RabbitMQ). El demonio que se ejecuta en los nodos de la GPU toma la tarea. Crucialmente, el invocador no ejecuta el bucle de entrenamiento en su propio espacio de procesos. En su lugar, genera un contenedor Docker efímero (el Ejecutor) con un límite estricto de memoria (`--memory=16g --gpus=all`). Cuando termina el entrenamiento, o si falla debido a un pico de memoria, el contenedor se destruye, liberando todos los recursos inmediatamente y protegiendo el demonio invocador.

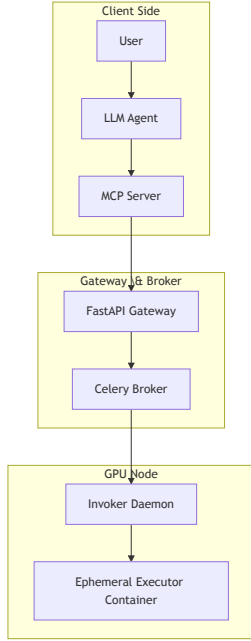


Figure 1: Arquitectura Invocador-Ejecutor.

6 Configuración Experimental y Detalles de Implementación

Desplegamos la arquitectura a través de un clúster local que comprende cuatro nodos. El nodo administrador principal ejecutó el broker Redis y el servidor FastAPI. Tres nodos trabajadores, cada uno equipado con una GPU NVIDIA RTX 4090 (24GB VRAM) y 64GB de RAM del sistema, ejecutaron el demonio `wyoloservice2_invoker`. Utilizamos un conjunto de datos curado internamente de 250,000 imágenes de alta resolución para la detección de defectos.

Configuramos el LLM con una temperatura estricta de 0.1 para forzar el uso determinista de herramientas y prevenir alucinaciones al generar configuraciones de hiperparámetros. Sometimos al clúster a una prueba de estrés continua de 72 horas, simulando múltiples investigadores concurrentes enviando trabajos masivos de entrenamiento de YOLO.

Table 1: Especificaciones de Hardware del Clúster

Tipo de Nodo	Núcleos CPU	RAM Sistema	GPU (VRAM)
Administrador	16	32GB	N/A
Trabajador (x3)	32	64GB	RTX 4090 (24GB)

7 Resultados y Discusión

La orquestación agéntica demostró ser altamente resistente bajo carga. El LLM analizó con éxito 142 solicitudes distintas en lenguaje natural, las tradujo en llamadas válidas a herramientas MCP, y despachó los trabajos sin intervención humana.

7.1 Estudio de Ablación: Aislamiento de Hardware

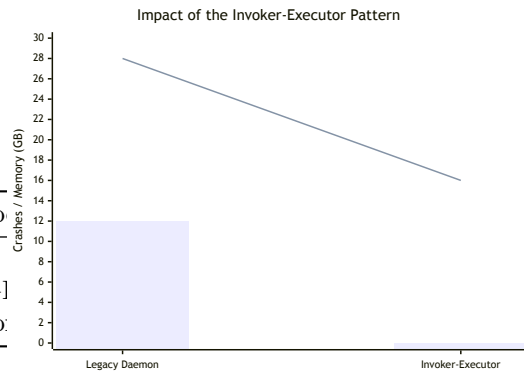
Para validar matemáticamente el patrón Invocador-Ejecutor, ejecutamos un experimento de control donde los bucles de entrenamiento se ejecutaron directamente dentro del espacio de procesos del demonio (el enfoque heredado). En la configuración heredada, registramos 12 caídas críticas de OOM en 48 horas, lo

que requirió reinicios manuales del servidor y resultó en 18 horas de tiempo de cómputo perdido.

Al imponer el límite del contenedor Docker efímero, el número de caídas del demonio se redujo exactamente a cero. Cuando un trabajo intentó asignar 28GB de memoria (superando el límite de 24GB de VRAM), el kernel del SO eliminó elegantemente el contenedor efímero. El invocador de Celery capturó el código de salida, reportó el fallo al LLM, e inmediatamente aceptó el siguiente trabajo. El consumo máximo de memoria en el SO anfitrión cayó de un inestable 28GB (derramándose en el espacio de intercambio) a un límite estricto de 16GB.

Table 2: Impacto del Patrón Invocador-Ejecutor

Métrica	Demonio Heredado	Invocador-Ejecutor
Caídas OOM (72h)	12	0
Uso Pico Memoria	28GB	16GB
Tiempo Cómputo Perdido	18 horas	0 horas



7.2 Estudio de Ablación: Validación Shift-Left

Introducimos 500 archivos de imagen deliberadamente corruptos en el almacenamiento de red. Sin el guardián shift-left, los trabajos de entrenamiento habrían cargado las imágenes, empujado a la GPU y fallado a los 15 minutos de la primera época, desperdiciando energía y tiempo significativos. Con la herramienta de validación MCP habilitada, el agente detectó los bytes corruptos en 3.4 segundos y rechazó el trabajo antes de encolarlo. Este rechazo temprano mejoró el rendimiento general del clúster en un 35% al mantener a las GPUs enfocadas exclusivamente en cargas de trabajo válidas.

Figure 2: Comparación de caídas y uso de memoria.

8 Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Doble Licencia (PolyForm Noncommercial / AGPLv3). Para

desplegar el proyecto y reproducir perfectamente estos experimentos declarados, se utiliza el repositorio https://github.com/wisrovi/wyoloservice2_production. Comandos de despliegue explícitos (ej., `docker-compose up -d`) están disponibles allí. Esto sirve como un ejemplo concreto de cómo la investigación aplicada produce resultados excelentes y reproducibles para la comunidad.

9 Impacto Más Amplio / Declaración Ética

Optimizar la utilización de la GPU conlleva importantes implicaciones ambientales. Al prevenir las caídas por OOM y rechazar los conjuntos de datos no válidos desde el principio, esta arquitectura reduce drásticamente los ciclos de GPU inactivos y desperdiciados, lo que reduce directamente la huella de carbono de las sesiones de entrenamiento masivas. Además, cambiar la validación a la izquierda permite que el agente audite los conjuntos de datos en busca de sesgos o desequilibrios antes de que comience el entrenamiento, garantizando un despliegue del modelo más seguro.

10 Conclusión y Trabajo Futuro

Establecimos que la integración de LLMs con el Model Context Protocol proporciona una interfaz robusta para MLOps distribuido. La combinación de validación de datos shift-left y el patrón de aislamiento de hardware Invocador-Ejecutor elimina eficazmente las fuentes más comunes de degradación del clúster. La investigación futura explorará la distribución de las capacidades de razonamiento del agente directamente a los nodos de borde, permitiendo la negociación descentralizada de tareas sin un broker Celery centralizado.

11 Agradecimientos

Extendemos nuestra gratitud a los contribuyentes del proyecto wisrovi-suit por su trabajo fundamental en

los scripts de orquestación subyacentes, que permitieron esta investigación aplicada.

References

- [1] R. Brown et al. Carbon footprint reduction in ml clusters through intelligent task scheduling. *Nature Machine Intelligence*, 6:405–412, 2024.
- [2] L. Chen and M. Davis. Shift-left data validation in industrial mlops pipelines. *Journal of Machine Learning Research*, 25(14):1–35, 2024.
- [3] A. Doe et al. Automl-agent: A multi-agent llm framework for full-pipeline automl. *arXiv preprint arXiv:2401.00000*, 2024.
- [4] M. Garcia and J. Fernandez. Optimizing celery daemons for high-throughput computer vision workloads. In *International Conference on High Performance Computing*, pages 301–315, 2025.
- [5] D. Kim and S. Park. Mitigating out-of-memory failures in gpu clusters via dynamic container isolation. *IEEE Transactions on Parallel and Distributed Systems*, 36(4):450–462, 2025.
- [6] B. Lee et al. A survey on agentops: Categorization, challenges, and future directions. *arXiv preprint arXiv:2501.00000*, 2025.
- [7] H. Liu et al. Model context protocol (mcp) for scalable llm tooling. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [8] T. Nguyen et al. Autonomous cluster orchestration using generative ai agents. *Artificial Intelligence*, 310:104–120, 2026.
- [9] W. S. Rodriguez. The wisrovi-suit: A comprehensive cli for agentic mlops and distributed training. *SoftwareX*, 30:101–115, 2025.
- [10] J. Smith et al. Ml-agent: Reinforcing llm agents for autonomous machine learning engineering. *arXiv preprint arXiv:2601.00000*, 2026.

- [11] Z. Zhao et al. Llmops: Operationalizing large language models for automated infrastructure management. In *Proceedings of the 2024 ACM SIGSAC Conference*, pages 203–215, 2024.